

Technikai interjú tudástar

Célzott felkészülés szakmai
kérdésekre és melyito follow-upokra

Kizárólag technikai kérdések és magyarázatok. Nincs
szakmai múlt, recruiter rész vagy motivacios sallang.

TypeScript - React - Node.js - REST - AWS - CI/CD -
Observability - LLM - MCP - AI agents
Telefonbarát kiadás - 2026. július

Tartalom

1. TypeScript	6 tema
2. React	11 tema
3. Node.js	7 tema
4. REST, HTTP és API design	6 tema
5. AWS és cloud architecture	6 tema
6. Testing, CI/CD és observability	4 tema
7. LLM, MCP és AI agent	5 tema
8. System design gyakorlatok	4 tema
9. Live coding és code review	5 tema

1. TypeScript

A pozíció erős TypeScript tudást kér. Nem a szintaxis a lényeg, hanem hogy a type system segítse a biztonságos Api-határokat, állapotmodellezést és refaktorálást.

Mi a TypeScript gyakorlati értéke egy nagy alkalmazásban?

LENYEGI VALASZ

A TypeScript sok integrációs hibát runtime helyett fejlesztés közben jelez. A legnagyobb értéke a közös contract: a komponens propok, API válaszok, domain modellek, eventek és konfigurációk ugyanazzal a szabálykészlettel ellenőrizhetők. A refaktorálást is biztonságosabbá teszi, mert a compiler megmutatja az érintett call site-okat.

FONTOS RESZLETEK

- A típusok nem helyettesítik a runtime validációt. A HTTP-ból, storage-ból, queue-ból vagy user inputból érkező adat továbbra is megbízhatatlan.
- A sok optional fieldet tartalmazó általános objektumok helyett domain-specifikus típusokat érdemes használni.
- A strict konfiguráció sokkal többet ér, mint a TypeScript egyszerű, laza annotációkkal használva.

VARHATO FOLLOW-UPOK

- Mély strict compiler opciók a legfontosabbak?
- Hol történjen runtime validation?
- Mikor válik túl bonyolultta a type system?

Mi a különbség az any, unknown, never és void között?

LENYEGI VALASZ

Az any kikapcsolja a type checkinget, ezért unsafe értékek terjedhetnek tovább a programban. Az unknown szintén bármit elfogad, de használat előtt narrowingot követel, ezért külső inputhoz jobb választás. A never olyan értéket jelöl, amely nem lehet, és jól használható exhaustive checkhez. A void azt jelenti, hogy a függvény visszatérési értéke nincs vagy szándékosan figyelmen kívül marad.

FONTOS RESZLETEK

- API- és error boundarynal az unknown legyen az alapertelmezett.
- Ha az any elkerülhetetlen, kis területre kell izolálni.
- A never segítségével egy új union case compile-time hibava tehető.

VARHATO FOLLOW-UPOK

- Miért unknown gyakran a catch-ben kapott error?
- Mi a különbség a never és az undefined-et visszaado függvény között?

Hogyan javítja a domain modellezést a discriminated union?

LENYEGI VALASZ

A discriminated union véges számú érvényes állapotot reprezentál egy közös taggel, például status vagy type mezovel. Minden állapot csak a hozzá érvényes mezőket tartalmazza. Így nem lehet olyan lehetetlen kombináció, hogy loading=true, miközben egyszerre data és error is van.

```
type Result<T> =  
  | { status: 'ok'; data: T }  
  | { status: 'error'; error: Error };  
  
function unwrap<T>(r: Result<T>): T {  
  switch (r.status) {  
    case 'ok': return r.data;  
    case 'error': throw r.error;  
    default: return assertNever(r);  
  }  
}
```

FONTOS RESZLETEK

- Különösen jó request state, workflow state, eventek és reducer actionok modellezésére.
- A discriminator alapján a switch pontos narrowingot ad.
- A neverrel készített exhaustive default jelzi, ha új case került be.

VARHATO FOLLOW-UPOK

- Hogyan modelleznéd egy shipment életciklusát?
- Miért veszélyes egyetlen interface sok optional fielddel?

Mitől jó egy generic absztrakció?

LENYEGI VALASZ

Egy hasznos generic valódi kapcsolatot fejez ki az értékek között, nem csak egy atnevezett placeholder. Például egy függvény olyan keyt fogad, amelynek leteznie kell az objektumon, és a megfelelő property típusat adja vissza. A constraint csak azt a minimális képességet írja elő, amelyre az implementacionak tenyleg szüksége van.

FONTOS RESZLETEK

- Az inference legyen erős, hogy a callernek ritkán kelljen explicit type argumentet megadnia.
- Kerülendo az a generic, amely csak elrejt az unknownt vagy egy castot.
- A publikus API ne tegye ki az implementacio bonyolult conditional type-jait, ha ez nem ad valódi biztonságot.

VARHATO FOLLOW-UPOK

- Mi a különbség a `K extends keyof T` és a `string` között?
- Mikor atlathatobb az overload, mint egy conditional type?

Hogyan kell biztonságosan tipizálni egy API választ?

LENYEGI VALASZ

Először transport schemát kell definiálni, majd runtime validacioval ellenőrizni a nyers választ, és csak ezután domain modellre alakítani. Egy TypeScript interface csak azt írja le, amit a fejlesztő vár; nem bizonyítja, hogy a szerver tenyleg azt küldte. Schema validation vagy generált client alakítja a külső adatot megbízható belső értékké.

FONTOS RESZLETEK

- A boundarynal egyszer validáljunk, ne az egész alkalmazásban újra és újra.
- DTO és domain object legyen külön, ha dátumot, pénzt, azonosítót vagy nullable mezőt kell konvertálni.
- A validation error tartalmazzon elég kontextust logolashoz és hibakereséshez.

VARHATO FOLLOW-UPOK

- Érdemes közvetlenül megosztani a típusokat frontend és backend között?
- Miben segít az OpenAPI code generation, és hol hibázhat?

Mély tsconfig beállítások fontosak production kodnal?

LENYEGI VALASZ

A strict az alap, mert bekapcsolja a szigorú ellenőrzések családját. A noUncheckedIndexedAccess jelzi az unsafe indexelést, az exactOptionalPropertyTypes megkülönbözteti a hiányzó propertyt az explicit undefined-tól, a useUnknownInCatchVariables javítja az error handlinget. A noImplicitOverride és a noFallthroughCasesInSwitches gyakori karbantartási hibákat fog meg.

FONTOS RESZLETEK

- Legacy codebase-ben a szigorítás fokozatosan vezethető be.
- A transpilation és a type checking legyen külön lépés, ha a bundler csak gyorsan transzpilál.
- A CI mindenképpen futtasson type checket.

VARHATO FOLLOW-UPOK

- Mi a különbség a transpilation és type checking között?
- Hogyan migrálnál fokozatosan egy JavaScript projektet?

2. React

Várhatóan a rendering, state ownership, effectek, performance és production viselkedés lesz fontos, nem csak a hookok szintaxisa.

Mitől renderel újra egy React komponens?

LENYEGI VALASZ

Egy komponens akkor renderel újra, ha a saját state-je változik, a parent újra renderel, egy felhasznált context value megváltozik, vagy egy external store subscription változást jelez. A render azt jelenti, hogy a React újra meghívja a komponenst és kiszámolja a következő element tree-t. Ez nem jelenti automatikusan, hogy a DOM is változik.

FONTOS RESZLETEK

- A React reconciliation során hasonlítja össze az új és előző tree-t.
- A child általában a parenttel együtt renderel, akkor is, ha a propok látszólag azonosak, hacsak memoization nem akadályozza meg.
- Development Strict Mode szándékosan többször is meghívhat renderhez kapcsolódó logikát.

VARHATO FOLLOW-UPOK

- Mi a különbség a render és commit phase között?
- Azonos értékkel hívott setState mindig DOM updatet okoz?

Hogyan működik a reconciliation, és miért fontos a key?

LENYEGI VALASZ

A reconciliation az elementeket type és pozíció alapján próbálja megfeleltetni a korábbi render elemeinek. Listában a key stabil identitást ad, így a React a megfelelő komponens state-et tartja meg akkor is, ha elemek beszurodnak, torlodnek vagy helyet cserelnek. A key nem normal propként érkezik meg a komponenshez.

FONTOS RESZLETEK

- Stabil domain identifier legyen a key, ne array index, ha a sorrend változhat.
- A key szándékos megváltoztatása reseteli a teljes subtree-t.
- Duplicate vagy instabil key rossz elemhez kotheti a local state-et.

VARHATO FOLLOW-UPOK

- Mikor elfogadható az array index keyként?
- Hogyan lehet keyvel egy formot resetelni?

Mikor segít a memo, useMemo és useCallback?

LENYEGI VALASZ

A memoization akkor segít, ha egy render vagy számítás merhetően drága, és az inputjai stabilak. A `React.memo` kihagyhat egy child rendert, ha a propok azonosak. A `useMemo` egy kiszámított értéket, a `useCallback` egy function identityt cache-el. Egyik sem teszi automatikusan gyorsabba a kódot.

FONTOS RESZLETEK

- A memoizationnak is van `comparison`, memória és komplexitási költsége.
- A parent által minden renderben létrehozott új object vagy callback hatástalanává teheti a `React.memo`-t.
- Először profilozni kell; sokszor jobb optimalizáció a state közelebb helyezése a felhasználáshoz.

VARHATO FOLLOW-UPOK

- Mit jelent a `referential equality`?
- Miért ronthatja a kódot a túl sok `useCallback`?

Hogyan osztanad fel a state-et local state, context, client store és server-state cache között?

LENYEGI VALASZ

A state maradjon a lehetőségekhez képest lokális. A context jó viszonylag stabil, egy subtree-ben sok helyen szükséges adathoz, például theme vagy user summary. Client store akkor hasznos, ha komplex megosztott kliensoldali workflowt vagy cross-cutting state-et kell koordinálni. A távoli rendszer által birtokolt adat server state, amelyhez freshness, refetch, deduplication és invalidation logikát ismero cache való.

FONTOS RESZLETEK

- A server datat ne masoljuk client store-ba egyértelmű ok nélkül.
- A tartós business state és az ideiglenes UI state legyen külön.
- Egy store koordinacios problémát old meg, nem rossz domain határokat.

VARHATO FOLLOW-UPOK

- Mikor okozhat performance problémát a context?
- Mély state kerüljön az URL-be?

Melyek a leggyakoribb useEffect hibák?

LENYEGI VALASZ

Az effect feladata a React szinkronizálása egy külső rendszerrel, például subscriptionnel, timerrel, requesttel vagy imperative browser API-val. Gyakori hiba a derived state effectben számolása, dependency kihagyása, loop létrehozása és cleanup elfelejtése. Az effectnek helyesen kell működnie start, cleanup és újraindítás után is.

```
useEffect(() => {  
  const controller = new AbortController();  
  loadUser(id,  
    controller.signal).then(setUser).catch(handleError);  
  return () => controller.abort();  
}, [id]);
```

FONTOS RESZLETEK

- Derived value-t render közben kell kiszámítani, nem külön state-ben tárolni.
- Az elavult requestet abortálni vagy az eredményt figyelmen kívül hagyni kell.
- A dependency list minden felhasznált reactive értéket tükrozzon.

VARHATO FOLLOW-UPOK

- Hogyan alakul ki stale closure?
- Miért fut developmentben kétszer az effect setup és cleanup Strict Mode alatt?

Hogyan alakul ki stale closure Reactben?

LENYEGI VALASZ

Egy render alatt létrehozott function az adott render prop és state értékeit zárja magába. Ha később timer, subscription vagy async callback használja, régi értéket olvashat. A javítás a szándéktól függ: dependency bovités, functional state update, ref használata nem-render adathoz, vagy a subscription helyes újraépítése.

FONTOS RESZLETEK

- A dependency lintert nem szabad megertes nélkül elhallgattatni.
- A ref nem okoz rendert, de ki is kerüli a React reaktiv modelljet.
- A functional update ideális, ha az új state csak az előző state-tól függ.

VARHATO FOLLOW-UPOK

- Mikor helyes a useRef?
- Hogyan készítenel olyan intervált, amely mindig a friss state-et látja?

Hogyan akadályozod meg a request race conditiont?

LENYEGI VALASZ

A legutobbi requestnek kell birtokolnia az eredményt. Az előző request AbortControllerrel megszakítható, vagy request identifier alapján az elavult response figyelmen kívül hagyható. Egy server-state library a cancellation, deduplication, retry és cache key logikát általában konzisztensabban kezeli, mint egyedi effectek.

FONTOS RESZLETEK

- Minden response-t befolyasolo input kerüljön a cache keybe.
- A cancellation és a valódi failure külön állapot legyen a UI-ban és a logokban.
- Egy lassú régi response nem irhatja felül az új adatot.

VARHATO FOLLOW-UPOK

- Hogyan kezelnéd az optimistic updatet?
- Mi történjen két utkozo mutation esetén?

Mire való a Suspense és a transition?

LENYEGI VALASZ

A Suspense lehetővé teszi, hogy egy subtree jelezze: támogatott async resource-ra var, és közben fallback boundary jelenjen meg. A transition non-urgent update-kent jelöl egy változást, így a React az urgent interakciókat responsive-an tarthatja, miközben elokeszíti a következő UI-t. Ezek nem helyettesítik a cache-t és error handlinget.

FONTOS RESZLETEK

- A boundarykat úgy kell elhelyezni, hogy egy lassú rész ne uresitse ki az egész oldalt.
- Transition jó navigation, filtering vagy drága, nem azonnali view change esetén.
- A Suspense error boundaryval és támogatott data layerrel működik jól.

VARHATO FOLLOW-UPOK

- Mi a különbség egy loading boolean és Suspense között?
- Mikor segít a useDeferredValue?

Hogyan diagnosztizálsz egy lassú React oldalt?

LENYEGI VALASZ

Méréssel kell kezdeni. A React Profiler megmutatja a drága vagy sokszor renderelo komponenseket, majd meg kell nézni, miért renderelnek. A browser performance trace alapján kereshető long task, layout munka, network waterfall, nagy bundle és memória növekedés. A legnagyobb bizonyított bottlenecket kell optimalizálni, nem mindenhova memoizationt tenni.

FONTOS RESZLETEK

- Hosszú listát virtualizálni kell.
- A nagy bundle felosztható és a nem kritikus route vagy feature lazy-loadolható.
- Csökkenteni kell a state fan-outot és a render alatti drága munkát.
- Ellenőrizni kell, hogy nem a backend vagy network a valódi bottleneck.

VARHATO FOLLOW-UPOK

- Mik a Core Web Vitals mutatók?
- Hogyan derítenél fel subscription okozta memory leaket?

Mit fog meg egy error boundary, és mit nem?

LENYEGI VALASZ

Az error boundary a descendant tree render, lifecycle és constructor hibait fogja meg, és fallback UI-t mutathat. Nem fogja meg automatikusan az event handler, async callback, szerveroldali hibákat vagy a saját boundaryn belül keletkezo hibát.

FONTOS RESZLETEK

- Meaningful product területek köré érdemes boundaryt tenni, nem csak egyszer az app rootba.
- Az error report tartalmazzon route-ot, release-t, user contextet és correlation ID-t.
- Az async failure state-ben vagy a data layeren keresztül jelenjen meg.

VARHATO FOLLOW-UPOK

- Hogyan recoverelnél egy hibás lazy-loaded chunk után?
- Milyen adatot küldenél error monitoring rendszerbe?

Hogyan tervezel karbantartható komponens API-t?

LENYEGI VALASZ

Kis, explicit API kell, amely a komponens felelosseget tükrözi. Composition jó, ha a callernek strukturális rugalmasság kell. Sok boolean prop helyett variant vagy discriminated union tisztabban zárja ki az érvénytelen kombinációkat. A data orchestration és reusable presentation különválasztható, ha ettől javul a tesztelhetőség és ownership.

FONTOS RESZLETEK

- Az accessibility szemantika legyen a reusable komponensben.
- Csak szándékosan támogatott DOM propokat forwardoljunk.
- Absztrakció csak több valódi use case után készüljön.

VARHATO FOLLOW-UPOK

- Mikor hasznos a compound component pattern?
- Hogyan tesztelned a behavior implementation detail nélkül?

3. Node.js

Várható mélyítési pont az event loop, async hibakezelés, scaling, stream és operációs viselkedés.

Hogyan befolyasolja az event loop a Node.js alkalmazás tervezését?

LENYEGI VALASZ

A JavaScript callbackek egy fő szálon futnak, miközben sok I/O műveletet az operációs rendszer vagy a libuv kezel. Az I/O befejezése után a callback event-loop phase-ekben fut le. A Promise callbackek és más microtaskok magas prioritásúak, ezért egy hosszú microtask lanc a timereket és I/O-t is késleltetheti.

FONTOS RESZLETEK

- Request pathon nem szabad heavy CPU munkával vagy szinkron API-val blokkolni a fő szálat.
- I/O várakozással a concurrency hatékony, de egy processen belül a JavaScript végrehajtás soros.
- Event-loop delay méréssel felismerhető a telitodes.

VARHATO FOLLOW-UPOK

- Mi a különbség a process.nextTick és Promise microtask között?
- Miért valhat performance problémává a JSON parsing?

Mi a különbség a concurrency és parallelism között Node.js-ben?

LENYEGI VALASZ

Concurrency esetén több művelet van folyamatban, tipikusan I/O-ra varva. Parallelism esetén a munka egyszerre fut több CPU magon vagy processen. A Promise.all concurrencyt hoz létre, de a CPU-heavy JavaScriptet nem teszi párhuzamossá. Valódi CPU parallelismhez worker thread, több process vagy külső worker kell.

FONTOS RESZLETEK

- Az unbounded Promise.all tulterhelheti a databaset vagy külső API-t.
- Nagy collection feldolgozasakor concurrency limit kell.
- Hosszú munkát queue-ba érdemes tenni, hogy a request latency kiszámítható maradjon.

VARHATO FOLLOW-UPOK

- Mikor használnál worker threadet child process helyett?
- Hogyan implementálnál bounded concurrencyt?

Mi a stream és backpressure?

LENYEGI VALASZ

A stream inkrementalisan dolgozza fel az adatot ahelyett, hogy az egész payloadot memóriába töltené. A backpressure lelassítja a producert, ha a consumer nem bírja a tempot. Node.js-ben a pipe és pipeline koordinálja a flow-t, completiont és error propagationt.

FONTOS RESZLETEK

- Stream jó nagy upload, download, compression, parsing és proxyzás esetén.
- Manual flow controlnal figyelni kell a `writable.write` visszatérési értékere.
- A pipeline segít elkerülni az elveszett errorrt és incomplete cleanupot.

VARHATO FOLLOW-UPOK

- Mi történik, ha figyelmen kívül hagyjuk a backpressure-t?
- Hogyan streamelnél biztonságosan nagy database exportot?

Hogyan kezeld a hibákat async Node.js service-ben?

LENYEGI VALASZ

Az elvart domain és validation hibákat explicit kell kezelni, transport boundarynal megfelelő válaszra fordítani, és az eredeti cause-t megőrizni logolashoz. Az unexpected errorrt központilag kell capture-elni és requesttel korrelálni. Ha a process inkonzisztens lehet, kontrollált restart a biztonságos megoldás.

FONTOS RESZLETEK

- Unhandled rejection vagy uncaught exception után nem szabad csendben tovább futni.
- Az operational error, például timeout, más kategória, mint a programmer error.
- A structured error tartalmazzon stabil code-ot, safe public message-et, cause-t és metadata-t.

VARHATO FOLLOW-UPOK

- Le kell allnia a processnek uncaught exception után?
- Hogyan akadályozod meg, hogy sensitive detail kerüljön az API response-ba?

Hogyan skálázol horizontálisan egy stateful WebSocket servicet?

LENYEGI VALASZ

A connectionok egy-egy instance-hoz kötődnek, ezért cross-instance üzenethez közös broker vagy pub/sub layer kell. Az authenticationt és connection metadata-t konzisztensen kell kezelni, a reconnectnek pedig működnie kell akkor is, ha a kliens másik instance-ra érkezik. A sticky session csökkentheti a mozgásokat, de nem váltja ki a distributed state tervezését.

FONTOS RESZLETEK

- A durable business state ne a WebSocket processben legyen.
- Definiálni kell a delivery semanticsot: at-most-once, at-least-once vagy application acknowledgement.
- Kell heartbeat, idle timeout, connection limit és backpressure.
- A deployment vagy network recovery reconnect stormot okozhat.

VARHATO FOLLOW-UPOK

- Hogyan kerülöd el a duplicate message-et?
- Mély metrikák fontosak egy WebSocket fleetnél?

Mit kell tudnia a graceful shutdownnak?

LENYEGI VALASZ

Termination signal után a service ne fogadjon új forgalmat, legyen unready, egy határidőig fejezze be az in-flight requesteket, zárja le broker és database connectionjeit, flusholja a fontos telemetryt, majd lépjen ki. Long-lived connectionnel explicit drain vagy reconnect stratégia kell.

FONTOS RESZLETEK

- Az orchestrator termination grace periodje legyen hosszabb az alkalmazás drain deadline-janal.
- A shutdown legyen idempotens, mert több signal is jöhet.
- Nem varhat orokke; deadline után force exit kell.

VARHATO FOLLOW-UPOK

- Hogyan drainelned a WebSocket klienseket?
- Mi a különbség a liveness és readiness check között?

Hogyan veded meg a Node.js API-t overloadtól?

LENYEGI VALASZ

Több retegen kell limit: body size, timeout, rate limit, connection limit, bounded queue, database pool limit és kontrollált downstream concurrency. A felesleges munkát korán kell elutasítani egyértelmű valasszal, nem megvarni, amíg a latency és memória korlátlanul nő.

FONTOS RESZLETEK

- Fairness esetén per-tenant vagy per-credential limit kell.
- A circuit breaker megved a folyamatosan hibazo dependency újrahívásától.
- A bulkhead megakadályozza, hogy egy workload minden erőforrást elfogyasszon.
- Monitorozni kell queue depth, event-loop delay, memória és downstream latency értékeket.

VARHATO FOLLOW-UPOK

- Mi a load shedding?
- Hogyan választasz timeoutot service-ek között?

4. REST, HTTP és API design

A pozíció REST API-kat és business feature-öket említ. A szemantika, kompatibilitás, security és distributed failure handling fontos.

Mitől RESTful egy API?

LENYEGI VALASZ

A REST egy architectural style, amely resource-okra, uniform interface-re, stateless requestekre, megfelelő esetben cache-elhető response-okra és layered komponensekre épül. Gyakorlatban egy jó HTTP API resource-oriented URL-t, standard method semanticsot, értelmes status code-ot és evolvalható representationt használ, anélkül hogy a belső storage részleteit kitenné.

FONTOS RESZLETEK

- A REST nem egyszerűen JSON HTTP felett.
- A resource business concept, nem feltétlenül database table.
- A stateless azt jelenti, hogy a request tartalmazza a feldolgozashoz szükséges kontextust; nem tiltja a szerveroldali adatot.

VARHATO FOLLOW-UPOK

- Mi a különbség REST és RPC között?
- Mikor elfogadható action endpoint?

Mit jelent a HTTP method safety és idempotency?

LENYEGI VALASZ

A safe method szándékosan nem változtat szerverállapotot; a GET és HEAD safe. Az idempotens method többször végrehajtva ugyanazt a szándékolt eredményt adja; a PUT és DELETE definíció szerint idempotens, a POST általában nem. A valódi implementációnak a log, notification és duplicate downstream side effecteket is figyelembe kell vennie.

FONTOS RESZLETEK

- Retry-zott create vagy payment-szerű művelethez idempotency key kell.
- A PATCH a patch formatum és operation függvényében lehet idempotens.
- A kliens és proxy a helyes method semanticsra építi a retry és cache viselkedést.

VARHATO FOLLOW-UPOK

- Hogyan implementálnád az idempotency-key store-t?
- A második DELETE 404 valasszal továbbra is idempotens lehet?

Hogyan tervezz API error formatumot?

LENYEGI VALASZ

A response tartalmazzon stabil machine-readable error code-ot, safe emberi üzenetet, releváns field-level detailt és correlation ID-t. A kategóriák legyenek konzisztensen mappelve: invalid input, authentication, authorization, missing resource, conflict, rate limit, dependency failure és unexpected server error.

FONTOS RESZLETEK

- Stack trace, SQL vagy belső service detail nem kerülhet ki.
- A 4xx azt jelzi, hogy a kliensnek kell változtatnia; az 5xx szerver- vagy dependency hiba.
- 409 jó current-state conflict esetén, 422 semantic validationra használható, ha a konvenció konzisztens.

VARHATO FOLLOW-UPOK

- Mikor 400 és mikor 422?
- Hogyan dönti el a kliens, hogy retry-zon-e?

Hogyan paginálsz nagy és közben változó datasetet?

LENYEGI VALASZ

Az offset pagination egyszerű és engedi a page jumpot, de nagy offsetnel drága, valamint változó adatnál elemek maradhatnak ki vagy duplikálódhatnak. A cursor pagination stabil ordered keyt használ, és általában jobb feedhez vagy operational datasethez. A sorrend determinisztikus legyen, gyakran timestamp plus unique ID alapján.

FONTOS RESZLETEK

- A cursor legyen opaque token.
- A result order legyen explicit.
- A next cursort adjuk vissza, ne database internalt.
- A filtering és authorization a cursor logikai kontextusához tartozik.

VARHATO FOLLOW-UPOK

- Mi történik, ha több row azonos timestampmel rendelkezik?
- Mi a total count trade-offja?

Hogyan fejlesztész tovább egy API-t kliens torese nélkül?

LENYEGI VALASZ

Az additive change a legbiztonságosabb: új optional field, új endpoint vagy régi kliensek által figyelmen kívül hagyható capability. Breaking change a field törlése vagy átnevezése, jelentés vagy típus változtatása, optional field kötelezővé tétele, illetve validation vagy ordering assumption megváltoztatása. Verzionálni akkor kell, ha a compatibility nem tartható.

FONTOS RESZLETEK

- Egy új enum value is törhet olyan klienst, amely exhaustive handlinget feltételez.
- Removal előtt deprecation telemetry és migration window kell.
- Contract test és schema diff segít az accidental breaking change felismerésében.

VARHATO FOLLOW-UPOK

- Mindig safe egy új response field?
- Mi a URL-, header- és media-type versioning trade-offja?

Mi a különbség authentication és authorization között?

LENYEGI VALASZ

Az authentication megállapítja, ki vagy mi a caller. Az authorization eldönti, hogy az identitás végrehajthatja-e a konkrét műveletet a konkrét resource-on. Az authorizationt minden releváns operationnel a szerveren kell kikényszeríteni, lehetőleg közel a business rule-hoz, nem csak route vagy UI szintén.

FONTOS RESZLETEK

- Short-lived credential kell, és ellenőrizni kell issuer, audience, signature, expiry és token type értékeket.
- Resource ownership vagy policy check nélkül insecure direct object reference alakulhat ki.
- A CORS browser access control, nem authorization.
- Cookie auth esetén CSRF védelmet explicit kezelni kell.

VARHATO FOLLOW-UPOK

- Mi a különbség role és permission között?
- Hol kell tenant isolationt érvényesíteni?

5. AWS és cloud architecture

Az erős válasz nem service-nevek felsorolása, hanem availability, security, cost és operációs trade-offok indoklása.

Tervezz highly available web applicationt AWS-en.

LENYEGI VALASZ

Tipikus design: Route 53 és CloudFront az edge-en, Application Load Balancer több Availability Zone-ban, stateless application instance-ok private subnetekben. Az adat managed multi-AZ database-ben vagy megfelelő distributed store-ban van. A static asset S3-ba kerül. Autoscaling, health check, central log, metrics, trace, backup és infrastructure as code teszi teljessé a rendszert.

FONTOS RESZLETEK

- Public subnetbe csak a publikus load balancer és kontrollált ingress kerüljön.
- Az alkalmazás IAM role-t használjon static credential helyett.
- High availabilityhoz minden single-instance dependencyt ki kell venni, és a failovert tesztelni kell.

VARHATO FOLLOW-UPOK

- Hol terminálnád a TLS-t?
- Hogyan deployolnál zero downtime-mal?
- Mi változik multi-region recovery esetén?

Mikor választasz Lambdát container helyett?

LENYEGI VALASZ

A Lambda event-driven, bursty, rövid életű munkára erős, amikor minimális infrastructure management fontos. A container jobb long-running service, special runtime, kiszámítható magas throughput, persistent connection vagy nagyobb CPU-, memória-, network- és startup-kontroll esetén.

FONTOS RESZLETEK

- A teljes költséget a vart traffic pattern mellett kell összehasonlítani.
- Cold start elsősorban latency-sensitive és ritkán hívott pathnal érdekes.
- A Lambda concurrency kontroll nélkül tulterhelheti a downstream rendszert.
- Containerhez is jó managed orchestration, például ECS vagy EKS.

VARHATO FOLLOW-UPOK

- Mi a különbség provisioned concurrency és reserved concurrency között?
- Használnál Lambdát WebSockethez?

Mi a különbség SQS, SNS és EventBridge között?

LENYEGI VALASZ

Az SQS durable queue, amely leválasztja a producereket és workereket, és elnyeli a burstöt. Az SNS push-alapú fan-out több subscriberhez. Az EventBridge event bus routing rule-okkal, AWS és SaaS integrációkkal és event filteringgel. Kombinalhatók is, például EventBridge vagy SNS külön SQS queue-kba fan-outol.

FONTOS RESZLETEK

- Standard SQS at-least-once deliveryt ad, ezért a consumer legyen idempotens.
- Kell dead-letter queue, alarm és replay folyamat.
- A visibility timeout legyen hosszabb a normal processingnel, vagy hosszabbitsuk meg.
- FIFO csak akkor kell, ha ordering és deduplication indokolja a korlátokat.

VARHATO FOLLOW-UPOK

- Mi történik, ha a processing tulnyulik a visibility timeouton?
- Hogyan kezeled a poison message-et?

Mikor választasz RDS vagy Aurora helyett DynamoDB-t?

LENYEGI VALASZ

Relacios database kell, ha join, több kapcsolt rekordot érintő transaction, rugalmas query és erős relational constraint fontos. A DynamoDB akkor jó, ha az access pattern ismert, nagyon magas scale és alacsony latency kell, és az adat partition és sort key köré modellezhető.

FONTOS RESZLETEK

- DynamoDB design access patternnel kezdődik, nem normalizált entitykkel.
- A hot partitiont nagy volumenű keyek szétosztásával kell elkerülni.
- Az Aurora managed replicationt és failovert ad, de a schema és query design továbbra is fontos.
- Optimistic concurrencyhez DynamoDB conditional write használható.

VARHATO FOLLOW-UPOK

- Mi az eventual consistency?
- Hogyan modelleznéd a many-to-many kapcsolatot DynamoDB-ben?
- Mi okoz database connection stormot serverless workloadnál?

Magyarázd el a public subnetet, private subnetet, NAT gatewayt és security groupot.

LENYEGI VALASZ

A public subnet route table-je internet gatewayhez vezet; egy resource csak akkor érhető el internetről, ha publikus címe és engedélyező kontrollja is van. A private application subnet nem teszi közvetlenül publikussá az instance-okat. A NAT gateway outbound internet access-t ad private subnetből, unsolicited inbound nélkül. A security group stateful allow rule resource szintén.

FONTOS RESZLETEK

- Támogatott AWS service-ekhez VPC endpoint csökkenti a NAT költséget és privát kapcsolatot ad.
- A network ACL stateless subnet-szintű kontroll, általában durvabb másodlagos védelem.
- Least-privilege security group másik security groupra hivatkozzon, ahol lehet.

VARHATO FOLLOW-UPOK

- Miért private subnetben van általában a database?
- Mi a különbség internet gateway és NAT gateway között?

Hogyan tervezel resilience-t egy hibazo dependency köré?

LENYEGI VALASZ

Kell explicit timeout, amely rövidebb a caller teljes deadline-janal. Csak transient failure retry-zható, exponential backoff és jitter mellett, limitált próbálkozással. Circuit breaker kell, ha a folyamatos hívás karos, és külön resource pool, hogy egy dependency ne foglaljon le mindent. Fallback csak akkor jó, ha a degraded answer business szempontból érvényes.

FONTOS RESZLETEK

- A retry-k retékenként megsokszorozódhatnak, ezért legyen egy felelos retry layer.
- Side effectes operation retry-ja előtt idempotency kell.
- Async queue használható, ha nem kell azonnali completion.
- Metric kell dependency latency, error rate, saturation és circuit state értékekre.

VARHATO FOLLOW-UPOK

- Miért fontos a jitter?
- Mi a retry storm?
- Hogyan osztod fel a timeout budgetet?

6. Testing, CI/CD és observability

A technikai kor gyakran azt vizsgálja, hogy a kódot hogyan szállítod és üzemelteted productionben.

Mit tesztelj unit, integration, contract és end-to-end szintén?

LENYEGI VALASZ

A unit test gyorsan ellenőrzi a determinisztikus domain logikát. Az integration test valódi boundaryt vizsgal, például databaset, queue-t, framework wiringot vagy serializationt. A contract test ellenőrzi, hogy külön deployolt service-ek ugyanazt ertik request és response alatt. Az end-to-end test néhány kritikus user journeyt validál a teljes rendszeren.

FONTOS RESZLETEK

- A business rule lehetőleg UI és transport alatt legyen, hogy olcson tesztelhető legyen.
- Nem minden dependencyt érdemes mockolni; a teszt atmehet, miközben a valódi integration hibás.
- Deterministic test data és izolált parallel run kell.

VARHATO FOLLOW-UPOK

- Mitől brittle egy teszt?
- Mikor hasznos a snapshot test?
- Hogyan tesztelned a retry-t és timeoutot?

Mi a különbség blue-green, canary és rolling deployment között?

LENYEGI VALASZ

A rolling deployment fokozatosan cseréli az instance-okat és erőforrás-hatékony, de régi és új verzió együtt el. A blue-green két teljes environmentet tart fenn és traffic switchel vált, így gyors rollbacket ad magasabb költséggel. A canary kis traffic arányt ad az új verziónak, és csak jó metrikák mellett növel.

FONTOS RESZLETEK

- Database change legyen backward compatible, amíg több verzió együtt fut.
- Automated rollback meaningful service-level metricre épüljön, ne csak process healthre.
- Feature flag szetvalasztja a deploymentet a feature release-tól.

VARHATO FOLLOW-UPOK

- Hogyan deployolnál breaking database migrationt biztonságosan?
- Mély metrika állítana le a canary rolloutot?

Mi a különbség log, metric és trace között?

LENYEGI VALASZ

A log diszkrét eventet rögzít kontextussal. A metric numerikus viselkedést aggregál időben, és hatékony dashboardhoz és alarmhoz. A trace egy request utjat és idejét mutatja komponenseken keresztül. Együtt megmutatják, mi hibázott, mekkora körben, és hol ment el az idő.

FONTOS RESZLETEK

- Structured log kell correlation vagy trace ID-val.
- Service-level indicator legyen latency, traffic, error és saturation.
- High-cardinality label drágává teheti a metricet; részletes ID logban és trace-ben legyen.
- Business outcome-ot is instrumentálni kell, nem csak infrastruktúrát.

VARHATO FOLLOW-UPOK

- Mi az SLI, SLO és error budget?
- Hogyan trace-elsz asynchronous queue workflowt?

Hogyan debugolsz production latency regressiont?

LENYEGI VALASZ

Először a scope-ot kell meghatározni: mély endpoint, user, region, verzió és kezdési idő érintett. Össze kell vetni latency percentile-t, error rate-et, throughputot, saturationt, deployment eventet és dependency timingot. Trace-szel izolálható a lassú szegmens, majd log és resource metric bizonyítja a feltetelezett okot. Nagy impactnal először mitigálni kell.

FONTOS RESZLETEK

- Átlag helyett p50, p95 és p99 értékeket kell nézni.
- Queue growth, connection-pool exhaustion, event-loop delay, database plan és downstream timeout tipikus ok.
- Timeline kell, és rollback vagy mitigation után recovery verification.

VARHATO FOLLOW-UPOK

- Miért félrevezető az average latency?
- Mi a különbség root cause és contributing factor között?

7. LLM, MCP és AI agent

A hirdetés explicit említi az MCP-t, agenteket és orchestratiónt. Gyakorlati engineering szemlélet kell, nem buzzword.

Mikor használsz RAG-et fine-tuning helyett?

LENYEGI VALASZ

A retrieval-augmented generation request időben ad releváns külső tudást a modellnek. Ez a tipikus választás, ha a tényadat változik, enterprise dokumentumra kell hivatkozni, vagy a hozzáférés userenként más. A fine-tuning inkább a modell viselkedeset vagy stílusát változtatja, és ismétlődő task patternhez jó; friss tényadat betöltésére nem megbízható.

FONTOS RESZLETEK

- A RAG pipeline része parsing, chunking, indexing, retrieval, reranking és evaluation.
- Az authorization retrieval előtt vagy közben történjen, ne generation után.
- Representative test settel kell mérni, nem csak demo alapján.

VARHATO FOLLOW-UPOK

- Hogyan választasz chunk size-ot?
- Mi okoz gyenge retrievalt jó modell mellett is?
- Mikor jó egyszerre RAG és fine-tuning?

Mi a tool calling, és hogyan kell biztonságosan implementálni?

LENYEGI VALASZ

A modell kiválaszt egy deklarált toolt és structured argumentet general. Az alkalmazás validálja az argumentet, authorizationt és policyt ellenőriz, végrehajtja a toolt, majd kontrollált eredményt ad vissza. A modell nem kaphat közvetlen, korlátlan infrastruktúra- vagy database-hozzaferest.

FONTOS RESZLETEK

- A model output untrusted input.
- Strict schema, allowlist, timeout, size limit és idempotency kell.
- A read-only és destructive tool legyen külön, side effecthez erősebb confirmation vagy approval kell.
- Audit miatt a tool call rögzíthető, secret kiszivargása nélkül.

VARHATO FOLLOW-UPOK

- Mi történik, ha a modell nem letezo argumentet general?
- Hogyan kerülöd el a duplicate side effectet retry után?

Milyen problémát old meg az MCP?

LENYEGI VALASZ

A Model Context Protocol szabványosítja, hogyan fedez fel és hív meg egy AI alkalmazás külső toolt, resource-t és prompt-szeru capabilityt egy MCP serveren. Szétválasztja a host alkalmazást, MCP clientet és capability providert, csökkenti az egyedi integration kódot, és hordozhatóbbá teszi a toolokat AI hostok között.

FONTOS RESZLETEK

- Az MCP integration protocol, nem autonóm agent framework.
- A host továbbra is kontrollálja a trustot, user consentet, authenticationt, authorizationt és elérhető servereket.
- A server description és tool output untrusted content, prompt-injection kockázattal.
- Least-privilege, szűk scope-u server jobb, mint egy mindenhez hozzáférő server.

VARHATO FOLLOW-UPOK

- Mi a különbség MCP tool és resource között?
- Hol tárolod a credentialt?
- Hogyan izolálsz third-party MCP servert?

Mikor indokolt agent, és mikor jobb deterministic workflow?

LENYEGI VALASZ

Deterministic workflow kell, ha a lépés, branch és success criteria ismert. Agent akkor indokolt, ha a rendszernek bizonytalanság mellett kell tool vagy plan között választania, és a rugalmasság megeri a nagyobb költséget, latencyt és kiszámíthatatlanságot. Sok production rendszer hybrid: deterministic orchestration korulvesz egy kis model-driven döntést.

FONTOS RESZLETEK

- Iteration, time, token budget és tool permission legyen limitálva.
- A workflow state külső durable storage-ban legyen, hogy biztonságosan folytathato legyen.
- High-impact actionhoz human approval kell.
- Long-running business processhez explicit state machine jobb.

VARHATO FOLLOW-UPOK

- Hogyan allitod le az agent loopot?
- Mi kerüljön orchestration code-ba prompt helyett?

Hogyan evaluate-elsz és observe-olsz AI feature-t productionben?

LENYEGI VALASZ

Valós taskokból versioned evaluation set kell, konkrét criterionokkal. Mérendő a task success, groundedness, tool correctness, latency, cost, refusal quality és safety failure. Productionben trace-elni kell a model callt, retrieval resultot, prompt versiont, tool decisiont és final outcome-ot, privacy és retention kontroll mellett.

FONTOS RESZLETEK

- Automated gradert kalibrálni kell human reviewhoz.
- Külön kell választani model quality, retrieval, tool és orchestration failure-t.
- Változást gradual rollouttal és ugyanazon evaluation seten kell összehasonlítani.
- A feedback gomb önmagában nem elég quality metric.

VARHATO FOLLOW-UPOK

- Hogyan detektalsz prompt injectiont?
- Milyen adat nem kerülhet trace-be?
- Hogyan tesztelsz model upgrade-et?

8. System design gyakorlatok

Ezeket hangosan érdemes végigvezetni: requirements és scale, egyszerű architecture, failure handling, data model, majd evolution.

Tervezz logistics shipment-tracking platformot.

LENYEGI VALASZ

Tisztazni kell az event source-ot, update frequencyt, delivery guarantee, query patternt, tenant isolationt és latency elvarast. Egy jó design authenticated API-n vagy queue-n fogad carrier eventet, validálja és normalizálja, tárolja a current shipment state-et és immutable event historyt, majd publikál downstream consumernek. Read API adja az aktuális állapotot, WebSocket vagy push a közel real-time updatet.

FONTOS RESZLETEK

- Shipment vagy tenant alapján partitionálni kell, hot key nélkül.
- Az ingestion legyen idempotens carrier event ID vagy deduplication key alapján.
- Out-of-order eventhez event time, sequence rule és domain state validation kell.
- DLQ és replay tooling kell invalid vagy tartósan hibás eventhez.
- Mérendő ingestion lag, stale shipment, duplicate rate és notification delivery.

VARHATO FOLLOW-UPOK

- Hogyan skálázod 100-szoros event volumennel?
- Hogyan javítasz hibás carrier eventet?
- Meddig tartod meg az adatot?

Tervezz pricing recommendation servicet logistics környezetre.

LENYEGI VALASZ

Az offline feature preparation legyen külön az online recommendationtól. Historical shipment, route, capacity és outcome adat batch vagy streaming feature pipeline-ba kerül. Az online API validated feature-t olvas, modellt vagy rules engine-t hív, business constraintet alkalmaz, majd recommendationt ad explanation metadata-val és version ID-val.

FONTOS RESZLETEK

- Deterministic fallback kell model vagy feature store kiesésére.
- Rögzíteni kell a recommendationt, input versiont, final human decisiont és outcome-ot evaluationhoz.
- Shadow mode-ban kell kezdeni, mielőtt az ajánlas árat módosít.
- El kell kerülni a data leakage-et és time-correct training data kell.
- Monitorozni kell driftet, acceptance rate-et, margin impactot, latencyt és missing feature rate-et.

VARHATO FOLLOW-UPOK

- Synchronous vagy asynchronous legyen?
- Hogyan rollbackelsz rossz modellt?
- Hol legyen a business rule?

Tervezz MCP toolokat használó AI operations assistantot.

LENYEGI VALASZ

A host authenticálja a usert, és csak a role által engedélyezett toolt teszi elérhetővé. Retrieval releváns runbookot és operation contextet ad. A modell először read-only diagnostic MCP toolt hívhat, például service health, deployment history és log query. Restart vagy config change csak strict validationnel, audit loggal és általában explicit human approvalval történhet.

FONTOS RESZLETEK

- Az incident workflow state a modellen kívül legyen.
- Log, ticket és tool description prompt-injection forras lehet.
- A tool scope environment, service és action alapján legyen szűk.
- Minden recommendationhez structured evidence kell.
- Historical incidenten és adversarial inputon kell evaluate-elni production access előtt.

VARHATO FOLLOW-UPOK

- Hogyan akadályozod meg, hogy rossz servicet restartoljon?
- Mi történik, ha két tool ellentmond?
- Hogyan támogatod az incident handovert?

Hogyan modernizálsz fokozatosan legacy PHP alkalmazást?

LENYEGI VALASZ

Strangler approach kell: egy bounded capability azonosítása, stabil routing vagy API layer ele helyezése, majd a capability új service-be mozgatása, miközben a legacy a többi továbbra is kezeli. A data ownership legyen explicit, és kerülni kell a kontrollálatlan dual write-ot. Migration előtt observability és contract test kell, hogy a behavior összehasonlítható legyen.

FONTOS RESZLETEK

- Low-risk, high-value boundaryval kell kezdeni, nem teljes rewrite-tal.
- Anti-corruption layer fordítja a legacy conceptet az új domain modellre.
- Data migrationnel egy authoritative writer és lehetőleg async replication kell.
- Cutover előtt rollback és coexistence terv kell.
- A régi path csak usage és correctness verification után törölhető.

VARHATO FOLLOW-UPOK

- Hogyan kezeled a shared database table-t?
- Mi a big-bang rewrite legnagyobb kockázata?
- Hogyan teszteled a behavioral equivalence-t?

9. Live coding és code review

Nem puzzle-memorizálás a cél. Tisztázd a feltételezéseket, válassz tiszta data structure-t, kezeld a failure eseteket, és teszteld az edge case-eket.

Hogyan állj neki egy live coding feladatnak?

LENYEGI VALASZ

Kódolás előtt fogalmazd újra a problémát és tisztázd a contractot. Menj végig egy kis példán, azonosítsd a data structure-t és az invariánsokat, majd implementáld a legegyszerűbb helyes megoldást. Közben mondd ki a complexityt, a failure behaviort és a teszteseteket; ne csendben próbálj tökéletes kódot írni.

FONTOS RESZLETEK

- Tisztázd az inputot, outputot, sorrendet, mutationt és az error elvárásokat.
- Először a correctness a cél; csak a constraint alapján optimalizálj.
- Menj végig empty, normal és adversarial példán.
- Ha elakadsz, mondd ki a bizonytalanságot, és bontsd kisebb lépésekre a problémát.

VARHATO FOLLOW-UPOK

- Mi a time és space complexity?
- Mi változik nagyon nagy inputnál?
- Mit hardenelnél production használatra?

Implementálj bounded asynchronous concurrencyt.

LENYEGI VALASZ

A jó megoldás task listát és maximalis concurrencyt fogad, csak ennyi taskot indít el, és minden befejezett task után indítja a következőt. Előre tisztázni kell, hogy a result megtartja-e az input sorrendet, és egy hiba leállít mindent vagy itemenként gyűjtjük az eredményt.

```
async function mapLimit<T, R>(
  items: T[], limit: number, fn: (x: T) => Promise<R>
): Promise<R[]> {
  const out = new Array<R>(items.length);
  let next = 0;
  async function worker() {
    while (true) {
      const i = next++;
      if (i >= items.length) return;
      out[i] = await fn(items[i]);
    }
  }
  await Promise.all(Array.from(
    { length: Math.min(limit, items.length) }, worker
  ));
  return out;
}
```

FONTOS RESZLETEK

- A limit legalább egy legyen.
- Nem szabad minden Promise-t előre létrehozni, mert a munka azonnal elindulhat.
- Productionben cancellation és maximum queue size is kellhet.

VARHATO FOLLOW-UPOK

- Hogyan tartod meg a result ordert?
- Hogyan valosítod meg a fail-fast és collect-all modot?

Review-olj React komponenst, amely effectben fetch-el.

LENYEGI VALASZ

Ellenőrizni kell a complete dependency listet, hogy obsolete request felülírhatja-e a current state-et, coherent-e a loading és error state, és a request inkább server-state library tulajdona-e. Meg kell nézni az unmountot, gyors ID-valtozást és azt is, mi történik, ha development Strict Mode kétszer indítja az effectet.

FONTOS RESZLETEK

- Derived datat ne tároljunk második state variable-ben.
- Requestet abortálni vagy stale resultot ignore-olni kell.
- A cancellation és failure ne legyen osszemosva.
- A cache key minden response-t befolyasolo inputot tartalmazzon.

VARHATO FOLLOW-UPOK

- Hogyan tesztelned a race conditiont?
- Hol legyen a retry policy?

Review-olj shipmentet létrehozó API endpointot.

LENYEGI VALASZ

Ellenőrizni kell authenticationt és tenant authorizationt, schema validationt, request-size limitet, idempotencyt, transaction boundaryt, duplicate protectiont és error mappinget. A downstream notification vagy event nem publikálható úgy, hogy a database transaction meg nem committed. Database change és event publish koordinálsához transactional outbox kellhet.

FONTOS RESZLETEK

- A client által küldött tenant ID policy check nélkül nem megbízható.
- Stabil resource ID és megfelelő status code kell.
- A unique constraint legyen a duplicate elleni végső védelem.
- Correlation ID logolható, sensitive request adat nem.

VARHATO FOLLOW-UPOK

- Mikor 201, 202 vagy 409?
- Hogyan működik a transactional outbox?

Mély edge case-eket kell kimondani coding task közben?

LENYEGI VALASZ

Releváns esetben: empty input, invalid input, duplicate, ordering, nagyon nagy input, cancellation, timeout, partial failure, concurrency és memory use. Ezután ki kell mondani az elvárt complexityt és azt a behaviort, amelyre a caller támaszkodhat. A tiszta contract gyakran fontosabb, mint egy clever implementacio.

FONTOS RESZLETEK

- Kodolás előtt representative example kell.
- Az invariantokat explicit ki kell mondani.
- Teszt kell smallest, normal és adversarial case-re.
- Readable kód kell, és jelezni lehet, hol jön production hardening.

VARHATO FOLLOW-UPOK

- Hogyan tesztelsz time-based functiont?
- Mikor jobb egy egyszerű $O(n \log n)$ solution a bonyolult $O(n)$ -nel?

A kérdések a Redspher Full Stack Developer hirdetés stackjehez és egy senior full-stack technikai interjú reális mélységéhez igazodnak.